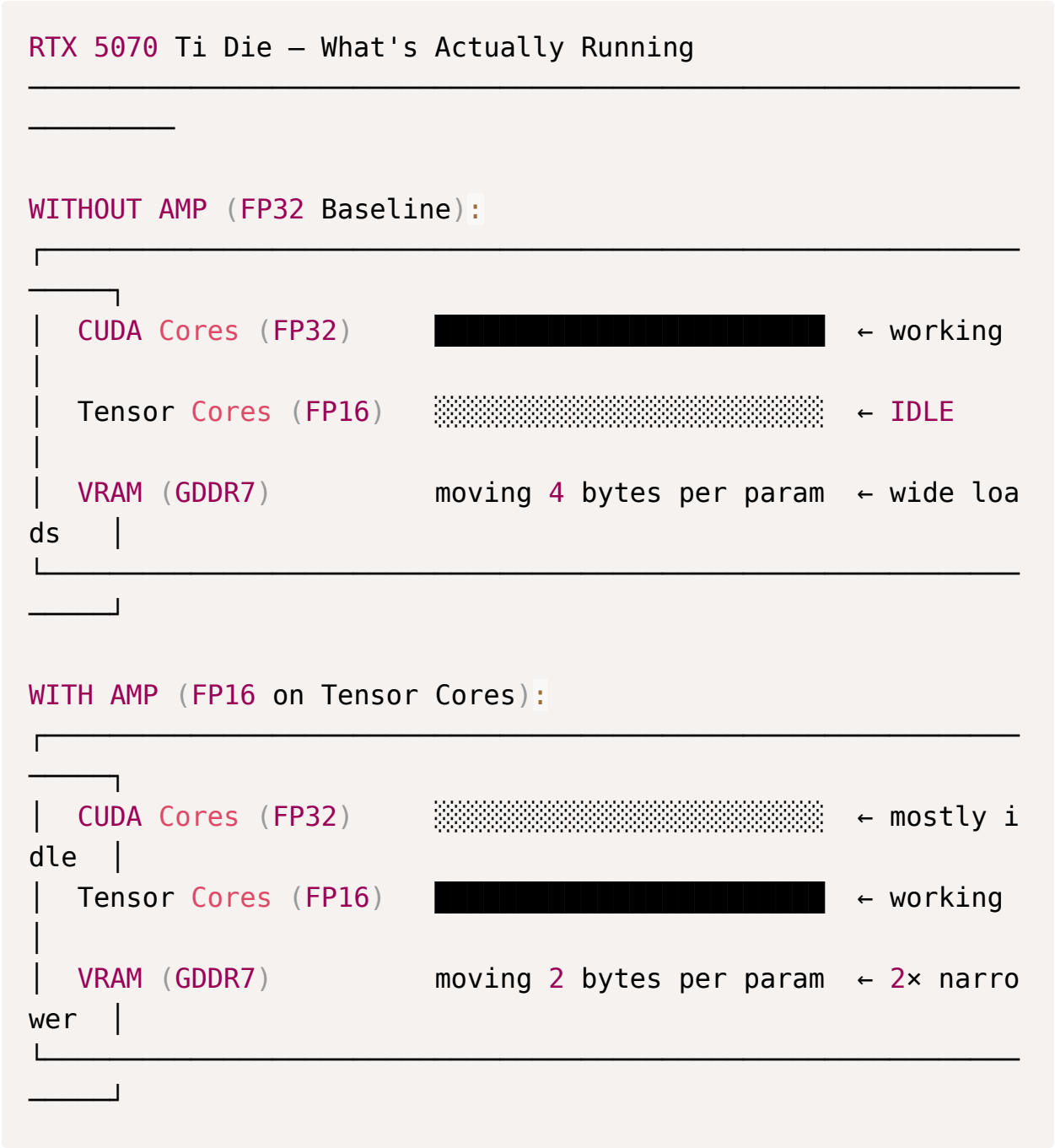
AMP

In baseline FP32 eager execution, the GPU CUDA cores do all the work. When AMP is enabled, it not only changes the number format, it also changes the compute route to a different piece of silicon that was not used before being Tensor Cores.



how its different

a standard CUDA core performed one multiply-accumulate (MAC) per clock-cycle, however a Tensor Core performs a 4 × 4 × 4 matric multiply-accu,ulate in a clock-cycle 64 MACS simultaneously. For teh U-Net convolution layers, which are fundamentally matrix multiplications (via `im2col`), this makes it a structural speedup.

```
CUDA Core (FP32) — 1 clock cycle:
──────────────────────────────────

  a × b + c = d        (1 MAC)

Tensor Core (FP16) — 1 clock cycle:
────────────────────────────────────

  ┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────┐
  │ 4×4     │ × │ 4×4     │ + │ 4×4     │ = │ 4×4     │
  │ matrix  │   │ matrix  │   │ FP32    │   │ FP32    │
  │ (FP16)  │   │ (FP16)  │   │ accum.  │   │ result  │
  └─────────┘   └─────────┘   └─────────┘   └─────────┘

  (64 MACs simultaneously)
```

Notice the accumulator stays **FP32**. This is not an accident — it's the central design decision of AMP, and it's why it doesn't destroy your segmentation accuracy.

time complexity change

in the baseline the time to compute a conv layer is dominated by:

```
T_baseline = (FLOPs) / (CUDA_core_throughput) + (memory_load_time_FP32)
```

under AMP it becomes:

```
T_AMP = (FLOPs) / (TensorCore_throughput) + (memory_load_time_FP16)
```

```
            ↑                                                    ↑
     ~16–32× faster                                  2× less data to
move
     for matmul ops
```

metrics (should probably use logged data from training)

```
Metric                      FP32 CUDA Cores      FP16 Tensor Co
res    Speedup
──────────────────────────────────────────────────────────────────
──────────────
Peak compute throughput     ~45 TFLOPS           ~1,406 TFLOPS
(sp.)  ~31×
VRAM bytes per parameter    4 bytes              2 bytes
2×
Effective memory BW use     896 GB/s (full)      896 GB/s (half
data) 2× efficient
PCIe transfer (201MB)       7.3ms (FP32)         3.65ms (FP16)
~2×
```

one thing to note

amp does not naively cast everything to FP16. it uses an autocast context manager that applies a precision policy for every kind of operation.

```
torch.autocast('cuda') precision policy:
──────────────────────────────────────────────────────────────────
────────

Operation Type           Precision    Reason
──────────────────────────────────────────────────────────────────
─────────
nn.Conv2d                FP16     ←── Tensor Core eligible, mat
mul-heavy
nn.Linear                FP16     ←── Tensor Core eligible
```

```
nn.BatchNorm2d          FP32    ←── Numerically sensitive, ne
eds range
torch.cat() (skip conn) FP32    ←── Accumulation — precision
required
Loss computation        FP32    ←── Cannot afford rounding in
gradients
Softmax                 FP32    ←── exp() unstable in FP16
```

contention and synchronization overheads under amp

Amp introduces a new synchronization cost that isn't included in teh baseline: **dtype casting at precision boundaries**.

```
AMP Execution Timeline — One Encoder Block:

CPU autocast:  [enter FP16 context]──────────────────────[exi
t → FP32]
                        |
|
GPU:            [Cast weights        [Conv2d          [BN in
[cast
                 FP32→FP16]           in FP16          FP32]
back]
                 ↑                   Tensor Core]      ↑
                 casting overhead    (fast)            prec
ision
                 ~microseconds                         boun
dary sync
```

this happens whenever an execution crosses a precision boundary fp16→fp32 or vice versa the CUDA runtime has to:

flush the tensor core pipeline, issue the cast kernel,resume in new precision

for the U-Net, each of the skip connections requires a cross precision form fp 16 to fp32, requirinf 4 additional cast kernels per forward pass that the baseline didnt have to worry about.

There is also the **GradScaler synchronization overhead,** AMP requires loss scaling to prevent FP16 gradient underflow:

```
GradScaler Workflow (per training step):
──────────────────────────────────────────────
① Scale loss upward:        loss × scale_factor  (prevents un
derflow)
② Backward pass in FP16
③ Unscale gradients:        grad / scale_factor
④ Check for Inf/NaN:        ← synchronization barrier — CPU m
ust read GPU result
⑤ If clean → optimizer.step()
   If overflow → skip step, reduce scale_factor
```

On the PCIe 4.0 x16 link, this device-to-host (D2H) readback — even for a single scalar — triggers a pipeline flush.